Cross-Team Operational Assistant

Phase-by-phase architecture and implementation guide

Version 1.0 | 13 October 2026 | Design proposal, not an implemented system

Deliverables: this editable Markdown source and a PDF with the same content. This guide specifies the approved application; it does not claim that any code, integration, account, or deployment already exists.

1. Requirements and scope

The application helps employees identify ownership, pending actions, dependencies, and delays in cross-team operational work. The first complete workflow is employee onboarding across HR, IT, Procurement, and the employee's manager. A RAG knowledge assistant answers questions from uploaded policies and procedures with source references.

Confirmed requirements

- Python is the main development language; the implementation must be understandable to a beginner.

- Provide a frontend, backend, and REST API endpoints testable independently in Postman.

- Use MongoDB Atlas with Atlas Vector Search, Mistral embeddings, Groq for answer generation, and AWS for deployment, following the latest selected stack.

- Implement document upload, extraction, chunking, embedding, retrieval, and answer generation as separately observable stages.

- Track operational requests, owners, missing information, task dependencies, approvals, deadlines, reminders, escalation, summaries, and audit history.

- AI assists classification, retrieval, and summarization. Rules and authorized humans control approvals and sensitive actions.

- Deliver architecture and phase-wise implementation details; implementation code and actual deployment are not requested here.

Design decisions for this first version

These are explicit proposed implementation choices, not claimed company requirements: one organization; manually maintained users and team ownership; text-based PDF ingestion; in-app notifications; one FastAPI codebase with separate modules; a background worker from the ingestion phase; a same-origin static frontend; read-only RAG answers; manually recorded completion of external actions.

Outside this version

Do not implement invoice, purchase-order, or customer-account workflows beyond the onboarding example. Do not integrate email, Slack, Teams, Salesforce, HR systems, or external ticketing tools. Do not actually grant application access or purchase laptops. Scanned-document OCR, Word ingestion, autonomous agents, React, LangGraph, hybrid search, reranking, streaming, and multi-tenant operation are excluded from the initial implementation. They are not prerequisites for this application.

Values not provided by the user

Real policy documents, team membership, approval rules, SLA durations, working-day calendars, exact Groq model ID, API quotas, AWS region, budget, and expected traffic were not supplied. Keep them configurable; do not manufacture policy facts or claim production sizing. Use clearly marked synthetic data for demonstrations. Ask the owner for these values when the corresponding implementation phase requires them. They do not block this architecture document.

2. Selected technology stack

Layer	Technology	Responsibility
Browser UI	HTML, CSS, Bootstrap, basic JavaScript	Forms, lists, dashboards; fetch JSON APIs
Application	Python, FastAPI, Pydantic	REST routes, validation, authorization, business rules
HTTP server	Uvicorn	Run the FastAPI application
Persistence	MongoDB Atlas, PyMongo	Operational records, document chunks, job records
Retrieval	Atlas Vector Search	Semantic similarity over authorized document chunks
Embeddings	Mistral mistral-embed	Embed document chunks and questions
Generation	Groq Python SDK	Call a supported configured text-generation model
Extraction	pypdf	Extract text and page references from text-based PDFs
Background jobs	Celery, Redis, Celery Beat	Ingestion, retryable processing, deadline checks
Testing	Postman, Pytest	Phase API verification and business-rule tests
Packaging	Docker, Docker Compose	Repeatable local application and worker setup
AWS	ECS Fargate, ECR, S3, ALB, Secrets Manager, CloudWatch	Containers, files, HTTPS routing, secrets, logs
Hosted queue	ElastiCache for Redis	Shared broker for API, worker, scheduler

Pin compatible stable dependency versions during setup and commit the lock file. This document does not prescribe unverified package versions or prices. Atlas remains a managed service separate from AWS application containers. Groq and Mistral are external API calls; deploying on AWS does not move their models into the AWS account.

3. System structure and boundaries

Component relationships

Source	Destination	Data or action
Browser or Postman	FastAPI	Authenticated REST request
FastAPI	Authorization and service modules	Validate actor, input, ownership, and state
Services	MongoDB Atlas	Persist records; run ordinary queries or vector retrieval
Ingestion service	Local file store / S3	Read original uploaded PDF
API / scheduler	Redis / Celery worker	Dispatch durable tracked jobs
Embedding adapter	Mistral	Send chunk text or question; receive embeddings
Generation adapter	Groq	Send bounded authorized evidence; receive answer
Worker	MongoDB	Save progress, outputs, notifications, failures
Browser	FastAPI job and status APIs	Poll progress; render completed results

Use one repository and a modular monolith: routers call services; services call repositories and provider adapters. Keep SDK calls out of route functions. The worker imports the same services so manual stages and full pipelines cannot drift into different implementations.

Suggested project organization

- app/main.py: application setup and router registration.

- app/api/: auth, documents, jobs, retrieval, answers, requests, tasks, approvals, admin, notifications.

- app/schemas/: Pydantic request and response contracts.

- app/services/: ingestion, retrieval, generation, workflow, access, audit.

- app/repositories/: MongoDB queries and transaction boundaries.

- app/providers/: Mistral and Groq adapters.

- app/workers/: Celery tasks and scheduler configuration.

- app/core/: configuration, logging, errors, authentication.

- frontend/: HTML pages, CSS, JavaScript; served by FastAPI.

- tests/: unit, API, and integration checks.

- postman/: collection and example environment without credentials.

- deployment/: Docker definitions and AWS configuration documentation.

RAG versus live operational data

Policy question: 'Which approvals are required for access?' Retrieve approved policy chunks and generate a cited explanation.

Status question: 'Why is onboarding delayed?' Read current request, task, dependency, and approval records using normal database queries. Generate an optional summary from those records, not from approximate vector matches.

Action: 'Approve access.' Require an explicit approval API call by the assigned authorized approver. A generated answer never executes this action.

4. Actors and access rules

Actor	Permitted operations
Employee/requester	Create own request, view permitted request progress, supply missing information, query permitted documents
HR/request coordinator	Coordinate assigned onboarding requests and create tasks from the configured template
Team owner/member	View assigned team work; update permitted tasks; report blockers
Assigned manager/approver	Record decisions on assigned approvals; see relevant request context
Operations manager	View authorized operational dashboards and escalation queues
Administrator	Manage users, teams, ownership configuration, workflow template, policy documents, and indexes

Roles alone are insufficient: check object-level access on every document, request, task, approval, retrieval result, and job. Derive permissions from the authenticated user; never trust role, team, or user IDs supplied as authorization claims in a request body.

For this learning implementation, use administrator-created accounts, a maintained password-hashing library with Argon2id, short-lived bearer tokens, and server-side account/role checks. Browser tokens stay in memory, not localStorage. Users sign in again on expiry; no refresh-token workflow is required in this version. Postman stores the token only in its local environment. Company SSO is not part of this first implementation. Authentication and permissions precede shared deployment.

5. Data model

Use stable string identifiers in public APIs; serialize MongoDB internal IDs consistently. Store timestamps in UTC and display them in the configured organization timezone. Examples use symbolic IDs, not existing records.

Collection	Essential fields
users	id, email, password_hash, roles, team_ids, active
teams	id, name, member_ids, lead_user_id
workflow_configs	id, version, onboarding task definitions, ownership rules, approval requirements, reminder settings
documents	id, version, title, file_hash, storage_key, status, allowed_team_ids, active_revision, created_by
document_chunks	id, document_id, revision, ordinal, text, page_start, page_end, embedding, embedding_model, dimensions, chunking_version, allowed_team_ids, active
jobs	id, type, actor_id, resource_id, input_revision, status, progress, attempts, result, error, timestamps
retrieval_runs	id, actor_id, query, chunk_refs, scores, configuration, created_at, expires_at
answers	id, actor_id, retrieval_id, text, citations, model_id, prompt_version, created_at
requests	id, requester_id, coordinator_id, category, details, status, missing_fields, config_version, version
tasks	id, request_id, owner_id, team_id, depends_on, requires_approval, status, due_at, blocker_reason, version
approvals	id, task_id, approver_id, status, decision_comment, decided_at, version
notifications	id, recipient_id, request_id, task_id, kind, read_at, dedup_key

Collection	Essential fields
audit_events	id, actor_id, action, resource_type, resource_id, prior_state, new_state, correlation_id, created_at
outbox_events	id, event_type, payload_reference, status, attempts, created_at

Define indexes for user email uniqueness, task owner/status/due_at, task request_id, document/revision/ordinal uniqueness, job actor/status, and notification dedup_key uniqueness. Use schema validation for required fields. Model references explicitly; MongoDB does not supply relational foreign-key enforcement.

Use a transaction for an approval decision, affected task update, audit event, and outbox event. Use optimistic concurrency with a version field; stale updates return 409. External API calls occur outside database transactions. Outbox dispatch is retried safely and consumers deduplicate events.

6. Document ingestion design

1. Validate authentication, file size, PDF signature, filename, and upload metadata. Only administrators manage knowledge documents in this version.

2. Save the file outside the public web directory locally; use a private S3 object in deployment. Save document metadata and a SHA-256 content hash.

3. Extract text page by page using pypdf. Retain page references. Empty/scanned or encrypted/unreadable PDFs produce an explicit unsupported/processing error; do not pretend OCR succeeded.

4. Clean repeated whitespace without removing meaningful headings or policy text. Keep original extraction output available for inspection.

5. Split text on paragraph/sentence boundaries with a configurable character-based target and overlap. A demonstration starting point is 2,000 characters with 200 overlap; this is tunable, not an optimized benchmark. Enforce the selected model's actual input limit separately.

6. Embed chunks in configurable batches. Save the exact model identifier and returned dimensions. Validate finite numeric vectors and consistent dimensions before writing.

7. Create the vector index once per supported embedding configuration, through an administrator-only operation. Index the vector field and access/revision filter fields.

8. Mark embeddings as saved separately from search readiness. Check index state and bounded visibility checks before marking the document searchable. A timeout is an observable pending/error condition, not a fabricated success.

Document progression: uploaded, extracted, chunked, embedded, searchable. Track failures in jobs with the last completed stage. Rerunning a stage creates a new processing revision or safely reuses an identical successful result. Do not mix old chunks with a new embedding configuration. Replacing source content requires a new version and invalidation of downstream outputs.

Avoid duplicate chunks through a unique document/revision/ordinal key. A repeat upload with the same content and access configuration may return the existing document reference. Different access classifications must not accidentally share unrestricted retrieval state.

7. Retrieval and answer design

Retrieval

Validate question length and top_k. For the demonstration use top_k=5 with a configurable maximum of 20; these are proposed defaults. Embed the question with the same Mistral model/configuration used for document chunks. Never mix embedding spaces even when dimensions happen to match.

Apply server-derived access filters and active revision filters in Atlas Vector Search. Return ordered chunks with their relevance scores, document ID, title, version, page range, and chunk ID. Scores are similarity measures, not probabilities or confidence percentages. Persist a retrieval run so generation can be tested independently.

Generation

Load a retrieval run owned by the caller and recheck current access and document validity. Build a bounded context containing question, source labels, and relevant passages. Treat document text as evidence, not instructions. Ask the LLM to answer only from the evidence and cite source labels. When evidence is insufficient, return an explicit insufficient_evidence result instead of inventing a policy.

Validate that cited IDs belong to the provided evidence. This does not prove factual grounding; evaluate answer support with a small human-reviewed question set. Record model ID, prompt version, retrieval ID, and latency without exposing secrets or document content in general logs. Truncate context at chunk boundaries to fit the configured model's input budget, reserving space for output.

The complete RAG endpoint invokes the same retrieval and generation services as the separate APIs. Missing provider configuration prevents generation with a clear error; task tracking remains available.

8. Operational workflow design

The configured onboarding template creates registration, access approval/provisioning, laptop procurement/configuration, and project allocation tasks as needed. These are demonstration task types, not an asserted company policy. The administrator must supply actual ownership and approval requirements.

Required request fields are validated against the active template. Missing information places the request in awaiting_information with a list of missing fields. AI may suggest category, services, urgency, or missing details, but the requester/coordinator confirms the result. The template determines actual tasks and mandatory approvals. Store the template version on each request.

Task states: pending, awaiting_approval, blocked, in_progress, completed, cancelled. Reject cycles and self-dependencies. A task cannot start or complete while required approvals are pending/rejected or required predecessor tasks remain incomplete. A cancelled predecessor does not automatically satisfy a dependency. Manual blockers require a reason and explicit resolution.

Approval states: pending, approved, rejected. Only the assigned active approver may decide a pending approval. Rejected approval blocks the associated work; resubmission creates a new approval record while preserving history. Approval does not itself grant real application access.

Request states: awaiting_information, open, in_progress, blocked, completed, cancelled. Derive progress from tasks; complete only when all mandatory tasks are completed. Closing or cancelling requests must be an authorized, audited action. Do not infer completion from a narrative summary.

The scheduler checks configured due_at values. For this version, deadlines are explicit UTC timestamps; do not invent working-day calculations. Create one notification per task/reminder milestone using a deduplication key. Escalate to the configured team lead or operations owner; absent routing configuration produces a visible configuration issue. Use in-app notifications only.

9. Common API contract

All business routes start with /api/v1 and use JSON except multipart file upload. API IDs below are illustrative. All endpoints require a bearer token unless explicitly marked public. Use OpenAPI documentation generated from Pydantic models as the executable contract during implementation.

Successful list responses use items and next_cursor. Apply server-side pagination and access checks. Use 201 for created resources, 202 for accepted jobs, and 200 for successful reads/updates. Every response carries a correlation ID header.

Error body example: {"error":{"code":"INVALID_STATE","message":"Required approval is pending","details":{"task_id":"task_demo"}},"request_id":"corr_demo"}.

Common errors: 401 missing/expired authentication; 403 denied action; 404 missing or inaccessible object; 409 stale version or invalid transition; 413 upload too large; 415 unsupported file; 422 invalid input; 429 rate limit; 502 provider failure; 503 unavailable dependency. Sanitize provider messages. Do not return stack traces or credentials.

For asynchronous work, respond with {"job_id":"job_demo","status":"queued","status_url":"/api/v1/jobs/job_demo"}. Job states are queued, running, succeeded, failed. Stage results and sanitized errors are exposed through the job endpoint. Configure bounded exponential retries only for transient failures. Use idempotency keys on job submissions and decision writes to avoid duplicate side effects.

Authentication and administration endpoints

Method and path	Input / output example	Access and validation
GET /health/live	200 {status: alive}	Public; no sensitive configuration
GET /health/ready	200 {status: ready} or 503	Minimal dependency result; detailed checks internal
POST /api/v1/auth/token	{email, password} -> {access_token, token_type, expires_in}	Public, rate limited; invalid credentials 401
GET /api/v1/users/me	-> {id, roles, team_ids}	Authenticated
POST /api/v1/admin/users	{email, initial_password, roles, team_ids} -> {id}	Admin; unique email, valid teams
PATCH /api/v1/admin/users/{id}	{active, roles, team_ids} -> updated user	Admin; audit permission changes
POST /api/v1/admin/teams	{name, member_ids, lead_user_id} -> {id}	Admin; users must exist
GET /api/v1/admin/teams	-> {items, next_cursor}	Admin
POST /api/v1/admin/workflow-configs	{task_definitions, ownership_rules, reminder_rules} -> {id, version}	Admin; validate dependency DAG and owners
GET /api/v1/admin/workflow-configs/{id}	-> stored configuration	Admin/coordinator

Document and pipeline endpoints

Method and path	Input / output example	Access and validation
POST /api/v1/documents	Multipart file, title, allowed_team_ids -> {document_id, status: uploaded}	Admin; size/signature/access rules
GET /api/v1/documents	-> {items, next_cursor}	Filter by caller permissions
GET /api/v1/documents/{id}	-> metadata and stage status	Permitted viewer
GET /api/v1/documents/{id}/source	-> authorized download	Recheck access; private file only
POST /api/v1/documents/{id}/extractions	{revision} -> job	Admin; uploaded source required
GET /api/v1/documents/{id}/extractions/latest	-> {pages:[{page,text}], revision}	Permitted viewer; paginated
POST /api/v1/documents/{id}/chunkings	{revision, target_chars, overlap_chars} -> job	Admin; extraction required; overlap < target
GET /api/v1/documents/{id}/chunks	-> {items:[{id,text,page_start,page_end}], next_cursor}	Permitted viewer; no vectors by default
POST /api/v1/documents/{id}/embedding-jobs	{revision} -> job	Admin; chunks required; model server-controlled
POST /api/v1/documents/{id}/ingestion-jobs	{} -> job	Admin; orchestrates extraction through readiness
GET /api/v1/jobs/{id}	-> {status, progress, result, error}	Owner/admin with resource access
POST /api/v1/admin/vector-indexes	{configured_index_name} -> {name,status}	Admin; allowlisted server configuration
GET /api/v1/admin/vector-indexes/{name}	-> {name,status,dimensions}	Admin

RAG endpoints

Method and path	Input / output example	Access and validation
POST /api/v1/retrieval-runs	{query,top_k:5} -> {retrieval_id,matches:[{chunk_id,score,source}]}	Authenticated; server-derived filters
GET /api/v1/retrieval-runs/{id}	-> saved retrieval with visible matches	Owner; recheck access and revisions
POST /api/v1/answers	{retrieval_id} -> {answer_id,status,text,citations}	Owner; reject expired/stale evidence
POST /api/v1/rag/queries	{query,top_k:5} -> {retrieval_id,answer_id,status,text,citations}	Same checks as individual stages

Operational endpoints

Method and path	Input / output example	Access and validation
POST /api/v1/request-analyses	{description} -> {suggested_category,fields,missing_fields}	Authenticated; advisory only
POST /api/v1/requests	{category:onboarding,details,config_id} -> {id,status,missing_fields}	Requester/coordinator; validate template
GET /api/v1/requests	-> {items,next_cursor}	Scope to caller visibility
GET /api/v1/requests/{id}	-> request, tasks, approval summary	Permitted participants
PATCH /api/v1/requests/{id}	{details,expected_version} -> updated request	Requester/coordinator; prevent unauthorized fields
POST /api/v1/requests/{id}/tasks	{title,owner_id,depends_on,due_at} -> {id}	Coordinator; valid owners and acyclic dependencies
PATCH /api/v1/tasks/{id}	{status,blocker_reason,expected_version} -> updated task	Assigned team/coordinator; transition rules
POST /api/v1/tasks/{id}/approvals	{approver_id} -> {id,status:pending}	Coordinator; configured approver rule
POST /api/v1/approvals/{id}/decisions	{decision,comment,expected_version} -> {id,status}	Assigned approver; pending only
GET /api/v1/requests/{id}/blockers	-> [{task_id,reason,owner_id,next_action}]	Participants; computed from current records
POST /api/v1/requests/{id}/summaries	{} -> {text,record_refs,as_of}	Participants; live records only
GET /api/v1/notifications	-> {items,next_cursor}	Recipient only
PATCH /api/v1/notifications/{id}	{read:true} -> {read_at}	Recipient only
GET /api/v1/dashboard	-> {pending,blocked,overdue,completed}	Counts scoped to caller
GET /api/v1/requests/{id}/audit-events	-> {items,next_cursor}	Authorized coordinator/manager/admin

Template-generated tasks are created by the request service once all required fields are available. The manual task endpoint supports authorized corrections; it must not duplicate template tasks. Workflow configuration is versioned, not silently edited for existing requests.

Concrete JSON examples

Retrieval request:

```
{"query":"What approval is needed for Salesforce access?","top_k":5}
```

Retrieval response shape (synthetic example; no actual policy content implied):

```
{"retrieval_id":"ret_demo","matches":[{"chunk_id":"chunk_demo","score":0.82,"source":{"document_id":"doc_demo","vers
ion":1,"page_start":2},"text":"Synthetic policy passage used for testing."}]}
```

Generation request and response shape:

```
{"retrieval_id":"ret_demo"}
```

```
{"answer_id":"ans_demo","status":"answered","text":"Example answer grounded in the supplied test passage
[1].","citations":[{"label":"1","document_id":"doc_demo","chunk_id":"chunk_demo","page_start":2}]}
```

Do not use these synthetic answer contents as fixtures asserting real policy facts. Build factual answer tests from the actual approved test documents.

10. Phase-by-phase implementation

Phase 0 - Project foundation

Goal: run a small Python API before introducing AI. Set up a virtual environment, dependency lock file, application structure, environment configuration, logging, and health routes. Create a Postman environment with base_url; keep secrets out of Git. Start a local MongoDB Atlas development connection and verify connectivity.

Deliverable: health endpoints and OpenAPI documentation. Postman checks: live=200; readiness reflects unavailable database. Exit gate: a fresh setup can start from the documented commands without credentials committed to source.

Phase 1 - Identity, permissions, and jobs

Implement administrator-created users, token issuance, role/object checks, team records, and configuration validation. Add job records, Celery worker, Redis broker, and idempotent dispatch through an outbox. All later processing uses this job framework. Start with one worker locally.

Endpoints: auth, users/me, admin users/teams, jobs. Postman checks: valid login; incorrect password; unauthorized document access; other user's job denied. Exit gate: roles are enforced server-side, and a queued test job completes or exposes a clear failure.

Phase 2 - Upload and extraction

Implement private storage abstraction (local now, S3 later), upload validation, hashing, document metadata, extraction worker, and extracted-page inspection. Limit input to text-based PDFs. Choose and document a configurable upload limit before implementation; do not treat an unchosen number as a requirement.

Endpoints: documents create/list/detail/source, extractions create/read, jobs. Postman checks: valid PDF, invalid signature, empty PDF, encrypted file, oversized file, repeat upload, denied access. Exit gate: readable page text is available, original files remain private, errors do not leave a false completed status.

Phase 3 - Chunking

Implement cleaning, boundary-aware character chunking, overlap validation, and revision tracking. Preserve page and source metadata. Store chunks without embeddings first so outputs can be inspected independently.

Endpoints: chunkings and chunks. Postman checks: extraction prerequisite; short and long documents; overlap bounds; rerun without duplicate records. Exit gate: chunks reconstruct representative policy content without losing attribution or mixing revisions.

Phase 4 - Embedding and indexing

Implement the Mistral adapter, configurable batching, provider input bounds, transient retries, and vector validation. Save model/dimensions metadata. Provision the vector index with permission/revision filter fields. Track saved and searchable states separately.

Endpoints: embedding-jobs, index administration, jobs. Postman checks: missing key, failed batch, resume/retry without duplicate chunks, dimension mismatch, index still building. Exit gate: a known test passage is retrievable after readiness; no claim of readiness based only on database insertion.

Phase 5 - Retrieval

Implement query validation, query embedding, authorized Atlas vector search, result serialization, and saved retrieval runs. Use ordinary Python services and PyMongo rather than introducing an orchestration framework.

Endpoints: retrieval-runs create/read. Postman checks: relevant question, unrelated question, empty corpus, invalid top_k, inaccessible policy, outdated revision. Exit gate: relevant test passages are returned and restricted passages are never exposed. Record a small reviewed set of expected source passages; tune retrieval using evidence rather than arbitrary score thresholds.

Phase 6 - Answer generation and full RAG

Choose an available Groq model compatible with the account and required context size. Implement prompt building, context limits, source labels, unsupported-answer behaviour, citations, and provider error handling. Add full RAG and full ingestion orchestration by calling existing services.

Endpoints: answers, rag/queries, ingestion-jobs. Postman checks: answer supported by evidence; insufficient evidence; invalid citation; expired retrieval; provider timeout; injected instructions in document text. Exit gate: reviewed questions have supported answers or appropriate abstentions, and both separated and combined pipelines behave consistently.

Phase 7 - RAG frontend

Create login, document upload/status, extracted-text/chunk inspection, retrieval results, and question-answer screens. Use Bootstrap for layout and JavaScript fetch for REST calls. Poll jobs; display pending/error states and clickable authorized source references. Keep SDK keys entirely on the server.

Exit gate: the browser can complete the same authorized RAG journey verified in Postman. No direct browser connection to MongoDB, Mistral, or Groq. No fake progress or fabricated source references.

Phase 8 - Onboarding workflow

Implement the versioned template, request validation, missing-information loop, confirmed AI suggestions, task creation, owner assignment, dependencies, approval decisions, and audit history. Build the request/task screens. Disable AI classification gracefully when unavailable; manual form submission remains functional.

Endpoints: workflow configs, request-analyses, requests, tasks, approvals, blockers, audit-events. Postman checks: missing manager information; invalid owner; dependency cycle; approval by wrong person; double decision; stale update; completion attempted while blocked. Exit gate: synthetic onboarding proceeds from creation through authorized approval and completion with a complete audit trail.

Phase 9 - Reminders, summaries, and dashboard

Add one scheduler instance, explicit deadline checks, deduplicated in-app reminders, configured escalation recipients, scoped dashboard counts, and live-record summaries. Never infer current status from RAG.

Endpoints: notifications, dashboard, request summaries. Postman checks: overdue task produces one notification per configured milestone; resolved task stops reminders; failed summary leaves operations usable. Exit gate: scheduler restarts do not duplicate notifications and summary references match current records.

Phase 10 - AWS deployment and final verification

Containerize API/frontend, worker, and scheduler. Push images to ECR; run them on ECS Fargate. Put HTTPS ALB in front of API, private documents in S3, broker in ElastiCache, keys in Secrets Manager, and logs in CloudWatch. Configure secure network access to Atlas and outbound access to model providers. Keep worker and scheduler off the public HTTP path.

Deploy to a development environment first. Confirm actual AWS region, budget, credentials, and company-data approval before any real deployment. This document does not create cloud resources. Re-run the same Postman collection against the deployed base_url and validate document persistence after container replacement.

Exit gate: authorized end-to-end RAG and onboarding workflows pass; container restart does not lose files/jobs; health and logs are usable; secret values are absent from responses and browser assets.

11. Frontend-to-API mapping

Page	Main actions	API modules
Login	Authenticate; show expiry message	auth
Knowledge documents	Upload, run individual stages or full ingestion, inspect status	documents, jobs
Document inspection	Read extracted pages, chunks, source	extractions, chunks, source
Retrieval inspection	Query; show ranked passages and scores	retrieval-runs
Knowledge assistant	Ask question; show grounded answer and citations	rag/queries, answers
Onboarding requests	Create request; supply missing fields; inspect progress	requests, request-analyses
Task and approval detail	Update assigned work; record approval; see blockers	tasks, approvals, blockers
Dashboard and notifications	View permitted counts, overdue work, reminders	dashboard, notifications
Administration	Manage users, teams, template, index	admin routes

Stage buttons are learning/admin controls; regular employees use the question-answer and request screens. Show empty, loading, success, and error states. Hiding a button never substitutes for backend permission checks.

12. Reliability, security, and observability

Use private file storage, HTTPS in deployment, server-side authorization, escaped frontend rendering, bounded uploads and input sizes, and rate limits on login and model calls. Do not log passwords, tokens, full vectors, or confidential policy text. Never treat text inside retrieved documents as executable instructions.

Persist progress before acknowledging durable jobs. Use an outbox and idempotent workers; a crash between provider response and persistence may repeat a paid API call, so record attempts and avoid promising exactly-once external execution. Retry rate-limit/transient server failures within a bounded limit; do not retry invalid credentials or unsupported files indefinitely.

Observe request latency, provider latency/failures, queue age, failed jobs, searchable document counts, reminder delivery, overdue tasks, and denied actions. Correlate API request, job, document/request IDs, and provider operation without exposing secrets. Audit records are append-only at application level, not claimed tamper-proof storage.

Use backup and restore capabilities appropriate to the chosen Atlas/S3 configuration; test recovery before real use. Capacity, retention, SLA guarantees, costs, and recovery targets remain uncommitted until requirements and account options are known.

13. Acceptance scenario and test strategy

Use synthetic employee Demo Employee and synthetic documents labelled as test material. Do not use private employee records for initial development.

1. Admin uploads an example onboarding policy; each ingestion stage is visible and independently testable.

2. Authorized employee asks a policy question and receives an answer citing the relevant page. A user outside the allowed team cannot retrieve that passage.

3. HR creates an onboarding request; missing required manager details are requested before task instantiation.

4. Manager approves access through the decision API. IT records provisioning completion manually; no external account is created by this application.

5. Procurement remains pending. Laptop configuration is blocked by its dependency, and the blocker view identifies the actual owner and next action.

6. A configured deadline passes; one in-app reminder/escalation is recorded per milestone.

7. Procurement records completion; IT can then configure the laptop. Remaining required work is completed and request status becomes completed.

8. Audit history and summary agree with recorded actions. AI provider failure does not stop task updates or approvals.

Postman folders follow the implementation phases. Store response IDs into environment variables rather than hardcoding database IDs. Assert status codes, response schema, stage progression, and negative access checks. Pytest focuses on dependency rules, state transitions, idempotency, authorization, and provider failure boundaries. Run integration tests against a dedicated test database and synthetic fixtures. Include human review of grounded answers; fluent wording alone is not a passing test.

14. Configuration and implementation questions

Required secret configuration: MONGODB_URI, MISTRAL_API_KEY, GROQ_API_KEY, authentication signing secret. Non-secret configuration: database name, vector index name, EMBEDDING_MODEL, EMBEDDING_DIMENSIONS, GROQ_MODEL, chunk settings, upload limit, provider timeouts, retry limit, retrieval expiry, job broker address, and deployment region/bucket.

Derive EMBEDDING_DIMENSIONS from the selected model output and enforce consistency with the index. Do not allow callers to select arbitrary models or index definitions. Validate required settings at startup; expose only sanitized readiness results.

Ask only at the relevant implementation gate:

- Before real workflow configuration: What are the approved onboarding tasks, owners, approvers, and reminder deadlines?

- Before using actual knowledge: Which approved text-based PDFs may be uploaded, and which teams may read them?

- Before model integration: Which supported Groq model is available in the user's account, and what usage limit should the application enforce?

- Before AWS deployment: Which account/region and budget apply, and is sending the selected data to the model providers permitted?

No additional application features are implied by these questions. Development can begin with explicit synthetic fixtures and local configuration.

15. Technical references and verification boundary

These official references underpin the proposed integrations. Exact model availability, package compatibility, service limits, and pricing must be checked during implementation. No performance benchmark, guaranteed cost, or completed implementation is claimed.

- FastAPI features and API documentation: https://fastapi.tiangolo.com/features/

- MongoDB vector search overview: https://www.mongodb.com/docs/vector-search/

- MongoDB vector index configuration: https://www.mongodb.com/docs/vector-search/indexes/vector-search-type/

- Mistral text embeddings: https://docs.mistral.ai/studio/knowledge-rag/embeddings/text_embeddings

- Groq supported models: https://console.groq.com/docs/models

- Groq API compatibility: https://console.groq.com/docs/openai

- Celery broker documentation: https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/

- AWS ECS Fargate: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html